

B9IS130 – Web Development for Information Systems

Continuous Assessment Brief

Module Information

Module Title	Web Development for Information Systems
Module Code	B9IS130
Module Leader	Dr. Obinna Izima
Level	Level 9 (Master's Level)
Assessment Type	Project
Restrictions on Length	2000 words (minimum word count)
Assessment Title	Web application on a topic of your choice
Assessment Weighting	85%
Assessment Number	1 of 2
Assessment Mode	Individual
Issue Date	20/05/2026
Due Date	30/06/2026 (23:59 Hours)
Feedback Date	Hand in date plus 2 weeks
Submission Method	Online via Moodle

Assessment Overview

This assessment evaluates the student's ability to design, develop, deploy, and critically evaluate a modern full-stack web application within an information systems context. Students are expected to demonstrate technical competency in front-end and back-end development, database integration, cloud deployment, security implementation, and software engineering best practices. The assessment also evaluates the student's ability to justify technical decisions, critically reflect on the development process, and communicate solutions professionally.

Assessment Components

Component	Marks
Part 1: Web Application Development	70
Part 2: Technical Report	20
Part 3: Presentation and Demonstration	10

Part 1 – Web Application Development (70 Marks)

Students are required to independently design and implement a complete web-based information system or proof-of-concept application demonstrating postgraduate-level technical competency. The application should demonstrate clear evidence of requirements analysis, front-end and back-end integration, database management, API integration, security implementation, software engineering principles, and cloud deployment.

The use of JavaScript-based full-stack frameworks such as the MERN or MEAN stack is strongly preferred, and students are expected to implement their solutions using JavaScript

or JavaScript-based technologies. Projects developed without JavaScript-based implementation will be considered fundamentally limited in scope and may be deemed flawed in meeting the intended learning outcomes of the module.

Assessment Criteria

1. Requirements Analysis and Planning

Students should clearly define the problem domain, identify intended users and stakeholders, and produce both functional and non-functional requirements. The proposed solution should demonstrate evidence of research, feasibility analysis, and planning appropriate to a Level 9 project.

2. Front-End Development and UX Design

Students are expected to develop a responsive and accessible user interface using a modern front-end framework or library. The interface should demonstrate strong user experience principles including usability, consistency, accessibility, navigation clarity, readability, and responsive design. Projects should provide a professional user experience with clear workflows and intuitive interactions.

3. Back-End Development and Database Integration

Students must implement secure and maintainable server-side functionality supported by an appropriate database solution. The backend should demonstrate modularity, separation of concerns, CRUD functionality, and effective data management practices. Appropriate validation and error handling mechanisms are expected.

4. API and Web Service Integration

Students should demonstrate the ability to integrate and consume APIs or web services within their applications. The project should demonstrate understanding of client-server communication, data exchange formats such as JSON or XML, and appropriate handling of external data sources.

5. Security Implementation

Students are expected to implement appropriate security controls and demonstrate awareness of common web vulnerabilities. This includes authentication, authorization, input validation, password protection, secure session management, and mitigation of common threats such as SQL Injection and Cross-Site Scripting (XSS).

6. Software Engineering Practice and Code Quality

Students are expected to demonstrate the application of sound software engineering practices throughout the development process. This includes maintainable and modular code structure, separation of concerns, scalability considerations, effective use of version control, clear project organization, and the implementation of appropriate design patterns and architectural principles to support code quality, reusability, and long-term maintainability.

7. Cloud Deployment

The final application must be deployed to a publicly accessible cloud hosting platform. Students should demonstrate the ability to configure deployment environments and manage a production-ready application.

Part 2 – Technical Report (20 Marks)

Students are required to submit a professionally written technical report with a minimum of 2,000 words. The report should critically discuss the research process, system architecture, technology choices, security implementation, testing strategy, deployment process.

Referencing

All reports must be academically referenced using the **Harvard Referencing Style**. Students are expected to provide appropriate in-text citations and a complete reference list for all academic sources, technical documentation, frameworks, libraries, tutorials, datasets, APIs, and any external materials used throughout the project. Failure to appropriately reference sources may result in penalties under the institution's academic integrity and plagiarism policies.

Part 3 – Presentation and Demonstration (10 Marks)

Students must prepare a professional presentation and system demonstration between 10–15 minutes in duration. The presentation should clearly explain the problem being addressed, technical implementation, design decisions, and system functionality. Students are expected to demonstrate confidence, technical understanding, and effective communication skills.

Submission Requirements

You must submit the following via Moodle:

1. Technical Report (PDF or Word format)
2. Presentation must be done in class
3. GitHub repository URL
4. Deployed application URL

Failure to provide working links may negatively impact marks.

Marking Rubric

Part 1 – Web Application Development (70 Marks)

Assessment Area	Fail (<40%)	Pass (40–49%)	Merit (50–59%)	Distinction (60–69%)	High Distinction (70%+)
Requirements Analysis and Planning	Limited or unclear problem definition. Little evidence of research, stakeholder analysis, or project planning. Functional and non-functional requirements are poorly defined or missing.	Basic problem definition and some evidence of planning. Requirements identified at a superficial level with limited justification.	Clear identification of the problem domain and project objectives. Functional and non-functional requirements reasonably defined with some justification.	Strong evidence of research and planning. Requirements are clearly articulated, structured, and aligned to project goals. Demonstrates critical understanding of feasibility and scope.	Comprehensive and professionally presented analysis. Demonstrates excellent research, stakeholder awareness, feasibility analysis, and clearly justified requirements appropriate to postgraduate-level work.
Front-End Development and User Experience	Poorly designed interface with inconsistent navigation, weak responsiveness, poor readability, and little consideration for accessibility or usability.	Functional interface with limited UX consideration. Basic navigation and responsiveness evident but inconsistent.	Well-structured interface with reasonable responsiveness, navigation, and usability. Some evidence of accessibility and consistent design principles.	Strong UX/UI implementation demonstrating intuitive navigation, responsive layouts, accessibility considerations, and professional visual consistency.	Excellent UX/UI design demonstrating professional-quality interaction design, accessibility compliance, responsive optimization, clear workflows, and highly polished user

Assessment Area	Fail (<40%)	Pass (40–49%)	Merit (50–59%)	Distinction (60–69%)	High Distinction (70%+)
					experience throughout the application.
Back-End Development and Database Integration	Weak or incomplete backend implementation. Limited database functionality with poor application structure and little evidence of data management practices.	Basic backend functionality implemented with limited modularity and database integration. CRUD operations partially functional.	Functional backend and database integration demonstrating reasonable application structure, data handling, and CRUD functionality.	Well-designed backend architecture with strong modularity, validation, separation of concerns, and efficient database management practices.	Robust and scalable backend solution demonstrating professional software architecture, secure data management, efficient database design, maintainability, and strong engineering practices.
API and Web Service Integration	No meaningful API integration or incorrect implementation of web services. Limited understanding of client-server communication.	Basic use of APIs or web services with limited functionality and weak integration into the application workflow.	Appropriate integration of APIs and web services demonstrating functional data exchange using JSON/XML.	Effective integration of APIs and external services supporting core application functionality. Demonstrates good understanding of asynchronous communication and data	Sophisticated and well-implemented API architecture demonstrating secure, efficient, and scalable integration of web services with strong handling of data exchange and application

Assessment Area	Fail (<40%)	Pass (40–49%)	Merit (50–59%)	Distinction (60–69%)	High Distinction (70%+)
				handling.	workflows.
Security Implementation	No evidence of security implementation. Application vulnerable to common threats with little or no input validation or authentication mechanisms.	Basic security measures implemented with limited understanding of vulnerabilities and mitigation techniques.	Appropriate implementation of common security controls including authentication, validation, and secure handling of user data.	Strong implementation of security principles with evidence of secure coding practices, threat mitigation, session management, and awareness of industry standards.	Comprehensive security implementation aligned with industry best practices and OWASP principles. Demonstrates strong understanding of secure architecture, authentication, authorization, validation, and vulnerability mitigation.
Software Engineering Principles	Poor code organization with little evidence of modularity, maintainability, or software engineering practices. Minimal use of version control or design	Some evidence of modularity and organization. Basic software engineering practices applied inconsistently.	Reasonable application of software engineering principles including code organization, maintainability, and separation of concerns.	Strong application of design patterns, modular architecture, maintainable code structure, and professional development practices.	Excellent demonstration of software engineering principles with scalable architecture, clean code standards, reusable components, maintainability, and strong adherence to

Assessment Area	Fail (<40%)	Pass (40–49%)	Merit (50–59%)	Distinction (60–69%)	High Distinction (70%+)
	principles.				professional development methodologies.
Cloud Deployment	Application not deployed or deployment is non-functional. No evidence of deployment management or environment configuration.	Application partially deployed with limited functionality or deployment instability.	Successfully deployed application accessible online with core functionality operational.	Stable and professionally deployed application demonstrating good configuration management and deployment practices.	Fully functional production-ready deployment demonstrating reliability, scalability considerations, environment management, and professional cloud deployment practices.
GitHub Repository	Repository missing, incomplete, or poorly organized. Little evidence of version control practices or documentation.	Repository available with limited commit history and basic project organization. Minimal documentation provided.	Organized repository demonstrating reasonable commit history, project structure, and documentation.	Well-maintained repository with meaningful commits, professional organization, and clear documentation including setup instructions and project details.	Exceptionally professional repository demonstrating excellent version control practices, comprehensive documentation, structured commits, clear README, maintainable

Assessment Area	Fail (<40%)	Pass (40–49%)	Merit (50–59%)	Distinction (60–69%)	High Distinction (70%+)
					project structure, and evidence of continuous development workflow.

Part 2 – Technical Report (20 Marks)

Criteria	Fail (<40%)	Pass (40–49%)	Merit (50–59%)	Distinction (60–69%)	High Distinction (70%+)
Research & Analysis	Limited/no research	Basic research evident	Adequate analysis and justification	Strong research and critical evaluation	Excellent critical analysis with extensive supporting evidence
Technical Discussion	Poor explanations	Basic technical discussion	Clear explanations of implementation	Strong technical depth and justification	Advanced technical insight and critical evaluation
Structure & Presentation	Poorly structured with major errors	Basic organization	Clear and readable	Well-structured and professional	Exceptionally professional and academically rigorous

Part 3 – Presentation and Demonstration (10 Marks)

Criteria	Fail (<40%)	Pass (40–49%)	Merit (50–59%)	Distinction (60–69%)	High Distinction (70%+)
Delivery & Communication	Poor delivery and excessive reading	Basic communication skills	Clear presentation with moderate confidence	Engaging and confident delivery	Highly professional, polished, and engaging presentation
Technical Demonstration	Weak or incomplete demo	Basic functionality demonstrated	Functional demonstration	Strong technical walkthrough	Excellent technical demonstration with deep understanding

Criteria	Fail (<40%)	Pass (40– 49%)	Merit (50– 59%)	Distinction (60–69%)	High Distinction (70%+)
Questions & Discussion	Unable to answer questions	Answers limited questions	Answers most questions adequately	Confident responses	Comprehensive and technically strong responses

Academic Integrity

All submitted work must be the student's own original work. Students must appropriately reference external materials and comply with institutional academic integrity policies. Any use of external code, AI-generated content, or third-party resources must be clearly acknowledged and understood by the student.